PREPARATION GUIDE · ODOO 19



Odoo: workflows and development basics

A short visual journey to understand what a company does, then how Odoo turns it into code.

🔁 ERP workflows 🐍 Python / ORM 🧩 XML views

PREPARED BY

Ouchari Ibrahim

ERP course · ISID

No prerequisites: we start from zero

BEFORE YOU START

Your journey map

No prior knowledge is needed. The guide has two parts: first understand what the company does, then understand how Odoo is coded. Each section has its own code (A1, B3…) and fits on one or two pages.

🅐 PART 1 · Functional

Understand how a company works: sales, purchases, stock, invoices.

- **A1** — **The sales workflow** — CRM → Sales → Stock → Invoice
- **A2** — **The purchase workflow** — Request → Receipt → Vendor bill
- **A3** — **Documents, models and leaks** — Technical names and lost money

🅑 PART 2 · Development

Understand how Odoo is written: Python, models, rights, views.

- **B1** — **Python refresher** — classes, self, dict, f-string
- **B2** — **An Odoo module** — the box you install
- **B3** — **Models and fields** — Odoo's tables
- **B4** — **Access rights** — who may do what
- **B5** — **Calculations and business rules** — depends, constrains, onchange
- **B6** — **Overriding create() and write()** — create, write, super
- **B7** — **Searching data** — search and domains
- **B8** — **Views** — XML, XPath, menus
- **B9** — **Mini-project** — putting it all together

🧭 How to read it?

KEY POINT
The idea to keep.

ANALOGY
A real-life picture.

TRAP
A classic mistake.

TRY IT
Something to do in Odoo.

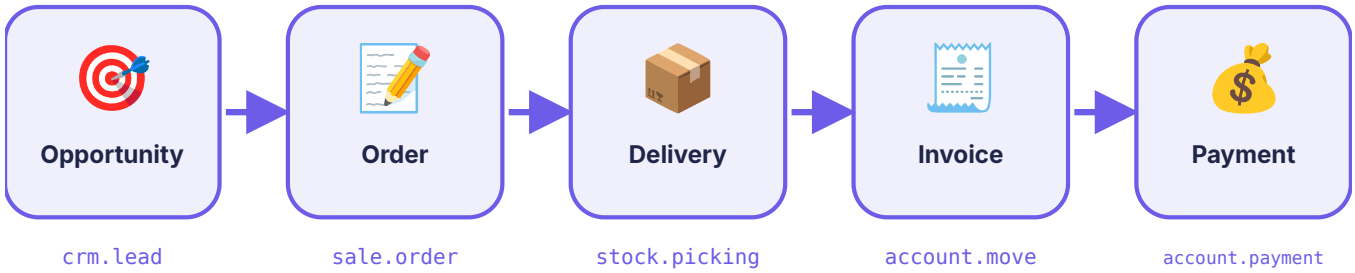
PART 1 · FUNCTIONAL

A1 The sales workflow

From the first contact to the customer's payment

ANALOGY: THE RESTAURANT

The customer orders (**Sales**), the kitchen prepares (**Stock**), the till charges (**Invoicing**). If the kitchen serves without telling the till, the restaurant loses money. Odoo links all these steps so nothing is forgotten.



Step by step

1. **An opportunity arrives (CRM module)**
   A prospect is interested in your products. You follow it in a visual pipeline.
   📍 CRM → Pipeline
2. **You send a quotation**
   From the opportunity, click **New quotation**: the customer is already filled in. Odoo computes the totals.
   📍 Sales → Quotations
3. **You confirm: the quotation becomes an order**
   Odoo creates the delivery order in Stock **by itself**. Nobody re-types anything.
   📍 Confirm button
4. **You deliver**
   The warehouse validates the delivery: stock decreases and the order becomes “to invoice”.
   📍 Inventory → Transfers
5. **You invoice, then the customer pays**
   Create the invoice from the order. The payment is matched with the invoice.
   📍 Invoicing → Customers

TRY IT

In Odoo: **Sales → Quotations → New**, add a product, click **Confirm**. A **Delivery** button appears at the top: the workflow has started.

PART 1 · FUNCTIONAL

A2 The purchase workflow

From asking the supplier to paying the bill

ANALOGY: THE SHOP THAT RESTOCKS

The manager sees the shelves emptying. They ask the wholesaler for a price, place the order, receive the boxes, check them, then pay the invoice. Odoo tracks each of these gestures.



Step by step

1. **Request for quotation**
   You pick the supplier and the products. Odoo can send the request by email.
   📍 Purchase → Orders → New
2. **You confirm: purchase order**
   The document becomes official and a **pending receipt** appears automatically in Inventory.
   📍 Confirm Order button
3. **You receive**
   When the boxes arrive, you validate the quantities really received: stock increases.
   📍 Inventory → Receipts
4. **Vendor bill**
   From the order, you create the vendor bill and compare it with the paper invoice.
   📍 Create Bill button
5. **Payment**
   On the due date, you register the payment. The debt is settled.

🔍 Three-way matching



✔ All three must match before paying

KEY POINT

Before paying, the accountant compares **the order, the receipt and the invoice**. If any one differs (quantity, price), there is a problem to solve before paying.

PART 1 · FUNCTIONAL

A3 Documents, models and leaks

Every document has a technical name, and every broken link loses money

🏷️ The technical name of each document

In code, a type of document is called a **model**. Here is the translation:

In real life	In Odoo (model name)
A customer or a supplier	`res.partner`
A product	`product.template` / `product.product`
A sales opportunity	`crm.lead`
A quotation or sales order (and its lines)	`sale.order` / `sale.order.line`
A delivery or a receipt	`stock.picking`
An invoice (customer or vendor)	`account.move`

🕳️ Where money leaks

When a link of the workflow breaks, the company loses money *without noticing*. Three typical leaks:



Delivered but not invoiced

The customer received the goods, but nobody sent the invoice.



Sleeping stock

Products bought that no longer sell: money stuck on a shelf.



Over-generous discount

A sale made at a loss because of a forgotten discount.

🔑 Two fields to know on a sales order

Field	Main values	What it says
`state`	`draft` → `sale` → `cancel`	Quotation, confirmed order, cancelled
`invoice_status`	`no` · `to invoice` · `invoiced`	Nothing to invoice · to invoice · already invoiced

KEY POINT

Spotting these leaks is a real need for companies. But before detecting them, you must first **understand the workflow** and the technical name of each document.

PART 2 · DEVELOPMENT

B1 Python refresher

The five Python tools you will meet all over Odoo's code

ANALOGY: THE COOKIE CUTTER

A **class** is a cookie cutter. Each cookie made with it is an **object**. And `self` means “this cookie”: the object we are working on right now.

1 · CLASS AND SELF

```
class Order:                          # the cutter
    def __init__(self, ref, total):
        self.ref = ref                # self = this order
        self.total = total

    def label(self):
        return f"{self.ref}: {self.total} TND"

o = Order('SO42', 4200)
print(o.label())                      # SO42: 4200 TND
```

2 · DICTIONARY, LIST AND LOOP

```
vals = {'name': 'Alert', 'amount': 500}   # dict
print(vals['amount'])                     # 500
vals['state'] = 'new'                     # add a key

orders = ['SO1', 'SO2', 'SO3']            # list
for ref in orders:
    print(ref)
```

3 · F-STRING (TEXT WITH VARIABLES)

```
name = 'Alert'
amount = 500
title = f"{name} : {amount}"
print(title)          # Alert : 500
```

4 · CONDITIONS (INDENTATION MATTERS!)

```
if amount >= 1000:
    level = 'medium'
elif amount >= 100:
    level = 'low'
else:
    level = 'none'
```

5 · LAMBDA

```
is_big = lambda o: o.total > 5000
# = a tiny function with no name
```

In Python	In Odoo, it is used for...
`class`	describing a model (a “table”)
`self`	pointing at the records the method works on
dictionary `{…}`	carrying the values to save (`vals`)
`for … in self`	handling records one by one
`f"…"`	building names and messages
`lambda`	writing a quick filter inside `filtered`

KEY POINT

Conditions are read **from top to bottom**: the first true one wins, and the following ones are ignored.

TRAP

In Python, the **spaces at the start of a line** (indentation, 4 spaces) say what belongs to the `if` or the loop. A wrong shift changes the whole meaning of the code.

PART 2 · DEVELOPMENT

B2 An Odoo module

The box you install to add a feature

ANALOGY: A LEGO BOX

A module is a box with a label (the manifest). Inside: data pieces (Python), screens (XML) and access rights (CSV). You install it, and Odoo assembles it by itself.

📁 A module's structure

MODULE FOLDER

```
risk_demo/
├── __init__.py          # imports models
├── __manifest__.py      # the ID card
├── models/
│   ├── __init__.py
│   ├── risk_alert.py
│   └── sale_order.py
├── views/
│   ├── risk_alert_views.xml
│   └── sale_order_views.xml
└── security/
    └── ir.model.access.csv # who is allowed
```

Folder	Content
`models/`	The Python: data and logic
`views/`	The XML: what the user sees
`security/`	The access rights

🪪 The manifest file

__MANIFEST__.PY

```
{
    'name': 'Risk Demo',
    'version': '19.0.1.0.0',
    'summary': 'Detect unbilled orders',
    'depends': ['sale_stock'],
    'data': [
        'security/ir.model.access.csv',
        'views/risk_alert_views.xml',
        'views/sale_order_views.xml',
    ],
    'license': 'LGPL-3',
    'application': True,
}
```

`depends` : the modules to install before ours.
`data` : the XML/CSV files loaded at install, **in this order** (security first, views next).

TRAP

An XML file that is **not listed in** `data`, or a missing line in an `__init__.py` : Odoo says nothing... and nothing shows up on screen.

KEY POINT

Install = load the module the first time. **Upgrade** (Apps → Upgrade) = reload after changing the code or the views.

PART 2 · DEVELOPMENT

B3 Models and fields

How Odoo stores information

ANALOGY: EXCEL

An Odoo **model** is like an Excel sheet: every column has a name and a type. Odoo creates the real database table for you (`risk.alert` becomes the table `risk_alert`).

📊 A model looks like a table

name (Char)	amount_at_risk (Float)	order_id (Many2one)	state (Selection)
Unbilled SO0042	4 200.0	SO0042	new
Unbilled SO0057	980.0	SO0057	approved

The **model** = the table · a **field** = a column · a **record** = a row

🧱 The most common field types

🔤 **Char** — short text

📄 **Text** — long text

🔢 **Integer / Float** — numbers

✅ **Boolean** — true / false

📅 **Date / Datetime** — dates

🔽 **Selection** — choice list

🔗 **Many2one** — link to 1

🧺 **One2many** — list of links

MODELS/RISK_ALERT.PY

```
from odoo import models, fields


class RiskAlert(models.Model):
    _name = 'risk.alert'
    _description = 'Risk Alert'

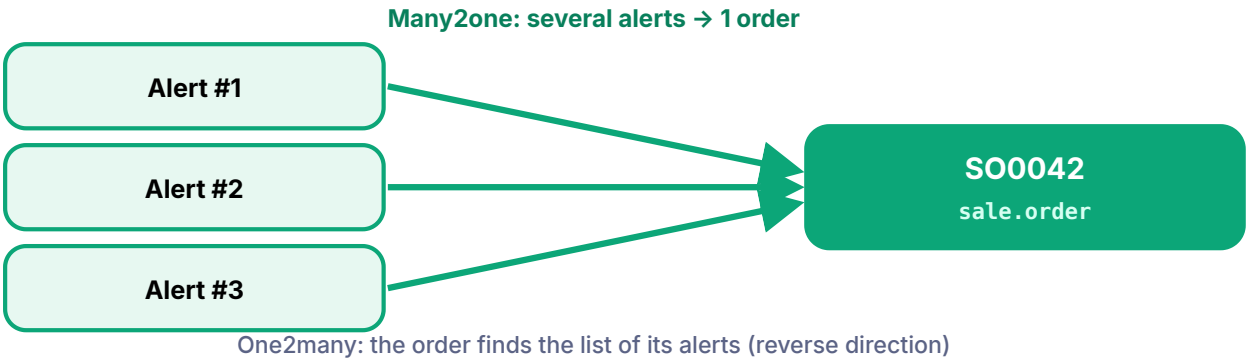
    name = fields.Char(string='Title', required=True)
    amount_at_risk = fields.Float(string='Amount at risk')
    order_id = fields.Many2one('sale.order', string='Sale order')
    state = fields.Selection(
        [('new', 'New'), ('approved', 'Approved'), ('done', 'Done')],
        default='new')
    closed_date = fields.Datetime()
    note = fields.Text()
```

PART 2 · DEVELOPMENT

B3 Models and fields

Linking models, and extending Odoo's own

🔗 Linking two models

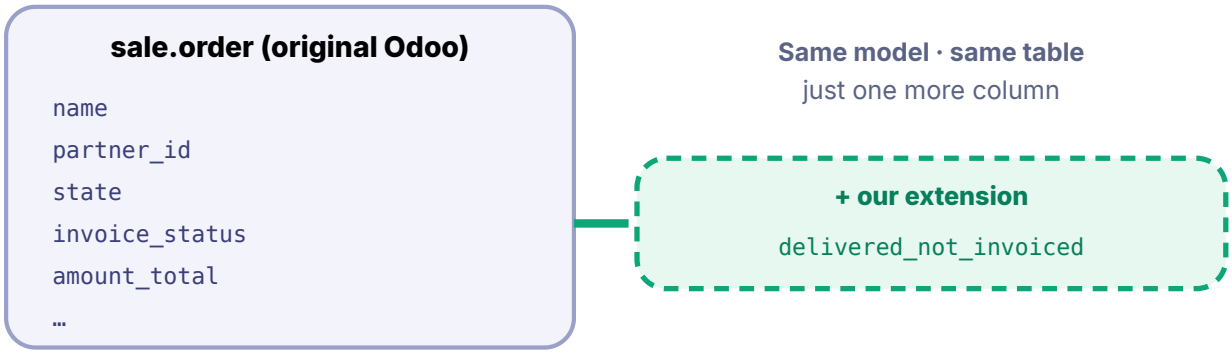


ONE2MANY: THE REVERSE LIST (2ND ARGUMENT = THE MANY2ONE FIELD ON THE OTHER SIDE)

```
alert_ids = fields.One2many('risk.alert', 'order_id')   # on sale.order: the reverse direction
```

🧬 Extending an existing model: `_inherit`

We may not (and need not) change Odoo's own code. Instead, we **add** our fields on top.



MODELS/SALE_ORDER.PY

```
class SaleOrder(models.Model):
    _inherit = 'sale.order'      # no _name: we extend the existing one

    delivered_not_invoiced = fields.Boolean(
        string='Delivered, not invoiced')
```

KEY POINT

`_name` = **create** a new model.
`_inherit` alone = **extend** an existing model: the original is not replaced, it gets new fields.

TRAP

Writing `_name = 'sale.order'` instead of `_inherit` : you try to recreate the model instead of extending it.

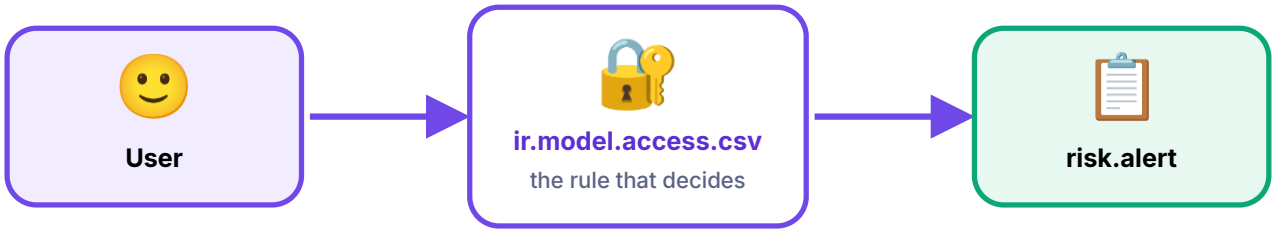
PART 2 · DEVELOPMENT

B4 Access rights

Who may see, edit, create or delete what?

ANALOGY: THE KEY TO THE DOOR

You just created a model: a new room in the house. But the door is locked. The file `ir.model.access.csv` hands out the keys and says what each person may do in the room.



Without a rule, the door stays closed for normal users

📄 One rule = one line in a CSV file

SECURITY/IR.MODEL.ACCESS.CSV

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_risk_alert_user,risk.alert.user,model_risk_alert,base.group_user,1,1,1,0
```

Column	What it means
`id`	The rule's unique name (your choice, but no duplicates)
`name`	A readable description
`model_id:id`	The model concerned: `model_` + its name with "_" instead of dots
`group_id:id`	**Who** gets the key. Here: all internal users (we won't detail groups today)
`perm_…`	Four switches: **1 = allowed**, **0 = forbidden**

🎚️ The four switches of the line above

1	1	1	0
perm_read	perm_write	perm_create	perm_unlink
see	edit	create	delete

KEY POINT

No rule = normal users get **no access** to the model. The file is listed in the manifest's `data`, **before the views**.

TRAP

As administrator, everything seems to work. The error only appears when **another user** tries: "You are not allowed to access…".

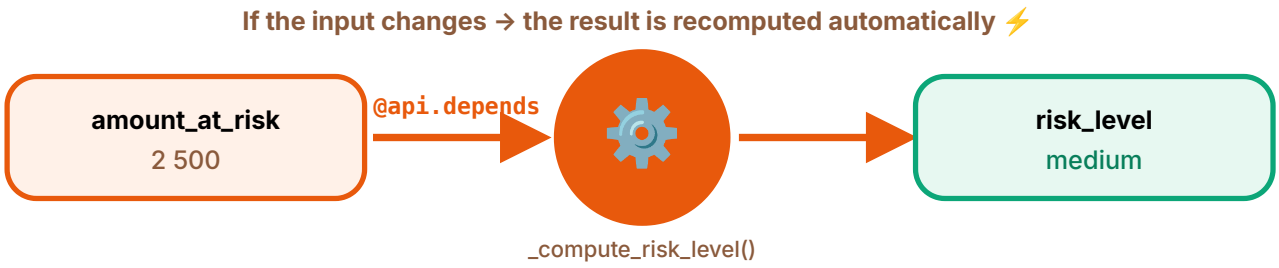
PART 2 · DEVELOPMENT

B5 Calculations and business rules

Fields that compute themselves: like an Excel formula

ANALOGY: THE EXCEL FORMULA

In Excel, a cell `=B2*C2` updates when B2 or C2 changes. An Odoo **computed field** does the same: we tell it which fields it depends on with `@api.depends`.



MODELS/RISK_ALERT.PY

```
risk_level = fields.Selection(
    [('low', 'Low'), ('medium', 'Medium'), ('high', 'High')],
    compute='_compute_risk_level', store=True)

@api.depends('amount_at_risk')
def _compute_risk_level(self):
    for rec in self:
        if rec.amount_at_risk >= 10000:
            rec.risk_level = 'high'
        elif rec.amount_at_risk >= 1000:
            rec.risk_level = 'medium'
        else:
            rec.risk_level = 'low'
```

KEY POINT

`compute='_name'` links the field to its method. `@api.depends(...)` lists the fields that trigger the recompute.

`store=True` : the value is **saved in the database** (you can search and sort on it). Without `store`, it is recomputed on the fly.

TRAP

Always **loop over** `self` (`for rec in self`) and **give the field a value in every case** (if / elif / else). Otherwise Odoo raises an error.

PART 2 · DEVELOPMENT

B5 Calculations and business rules

Forbid the wrong, help the user

🛑 Block invalid data

A **constraint** checks a business rule on save. If the rule is broken, we raise a `ValidationError`: the save is refused and the user sees the message.

MODELS/RISK_ALERT.PY

```
from odoo import api
from odoo.exceptions import ValidationError

@api.constrains('amount_at_risk')
def _check_amount(self):
    for rec in self:
        if rec.amount_at_risk < 0:
            raise ValidationError("The amount cannot be negative.")
```

💡 Help while typing

MODELS/RISK_ALERT.PY

```
@api.onchange('order_id')
def _onchange_order_id(self):
    if self.order_id:
        self.name = f"Check {self.order_id.name}"   # just a suggestion
```

⚖️ The three decorators side by side

	@api.depends	@api.constrains	@api.onchange
Role	Compute a value	Forbid invalid data	Help the user fill in the form
When?	When the listed fields change	On save	While typing, *before* saving
Result	Field value updated	Save **blocked** + message	Suggested value, **no blocking**

🎮 Which decorator to choose?

Situation
“The risk level depends on the amount”
@api.depends

Situation
“A negative amount is forbidden”
@api.constrains

Situation
“Pre-fill the title when an order is chosen”
@api.onchange

TRAP

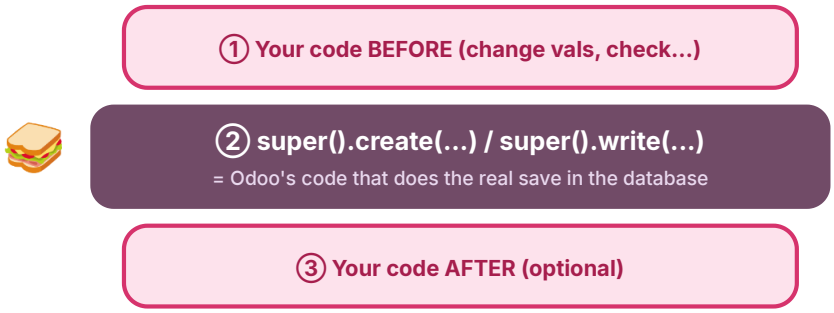
`@api.onchange` protects **nothing**: it only acts inside the form. For a real rule (that must hold even with an import or code), you need `@api.constrains`.

B6 Overriding create() and write()

Add your logic at the moment a record is created or modified

ANALOGY: THE SANDWICH

Overriding a method is making a sandwich: your code is the **top bread**, Odoo's code (`super()`) is the **filling**, and maybe some bread below.



➕ `create()` : on creation

```
CREATE
@api.model_create_multi
def create(self, vals_list):
    for vals in vals_list:
        if not vals.get('name'):
            vals['name'] = 'New alert'
    return super().create(vals_list)
```

✏️ `write()` : on modification

```
WRITE
def write(self, vals):
    if vals.get('state') == 'done':
        vals['closed_date'] = fields.Datetime.now()
    return super().write(vals)
```

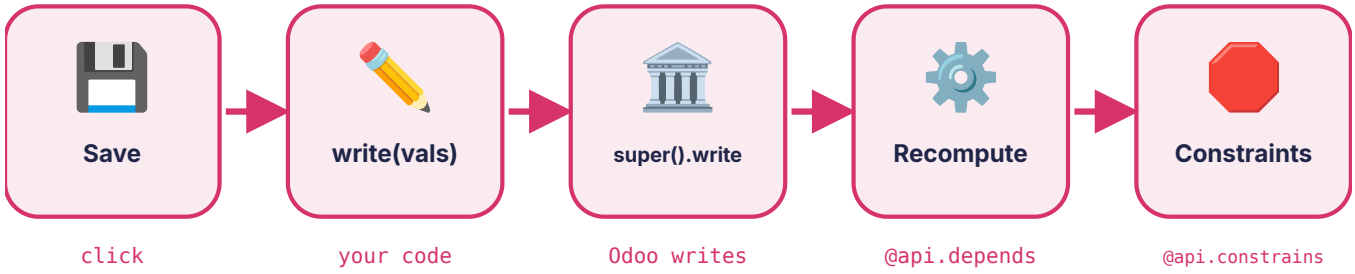
WHAT IS "VALS"?

A **dictionary** `{field: value}` : for `write` , what is being changed. For `create` , a list of dictionaries (one per record).

WHY SUPER()?

To **call the parent implementation**: without it, nothing is really saved.

⏱️ What happens when you click "Save"



TRAP

Forgetting `super()` (or forgetting the `return`): the record is not correctly created or modified, and often no error warns you. It is the #1 beginner mistake.

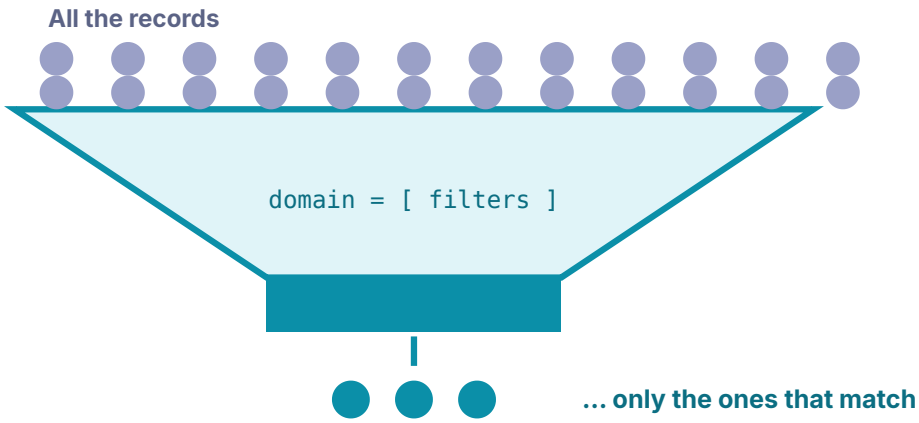
PART 2 · DEVELOPMENT

B7 Searching data

Query Odoo's database from Python

ANALOGY: AN EXCEL FILTER

Searching in Odoo is applying a **filter** to a table. The filter is called a **domain**; the result, a **recordset**.



🔍 The search tools

```python
Order = self.env['sale.order']           # the "table" of orders

orders = Order.search([('state', '=', 'sale'),
                       ('invoice_status', '=', 'to invoice')])

n      = Order.search_count([('state', '=', 'sale')])    # how many?
total  = sum(orders.mapped('amount_total'))              # sum of a column
names  = orders.mapped('name')                           # list of names
big    = orders.filtered(lambda o: o.amount_total > 5000) # filter in Python
new    = self.env['risk.alert'].create({'name': 'Alert', 'amount_at_risk': 500})
```

A domain = a list of conditions

Condition	Meaning
`('state', '=', 'sale')`	confirmed order
`('amount_total', '>', 5000)`	amount > 5000
`('name', 'ilike', 'SO')`	contains “so”
`('state', 'in', ['draft','sent'])`	either one

Several conditions in the list = **AND**. For an **OR**, put `'|'` in front (prefix notation).

A recordset = a collection of records

The result of a `search` is not a plain list: it is a **recordset**. You can loop over it, count, filter and extract columns.

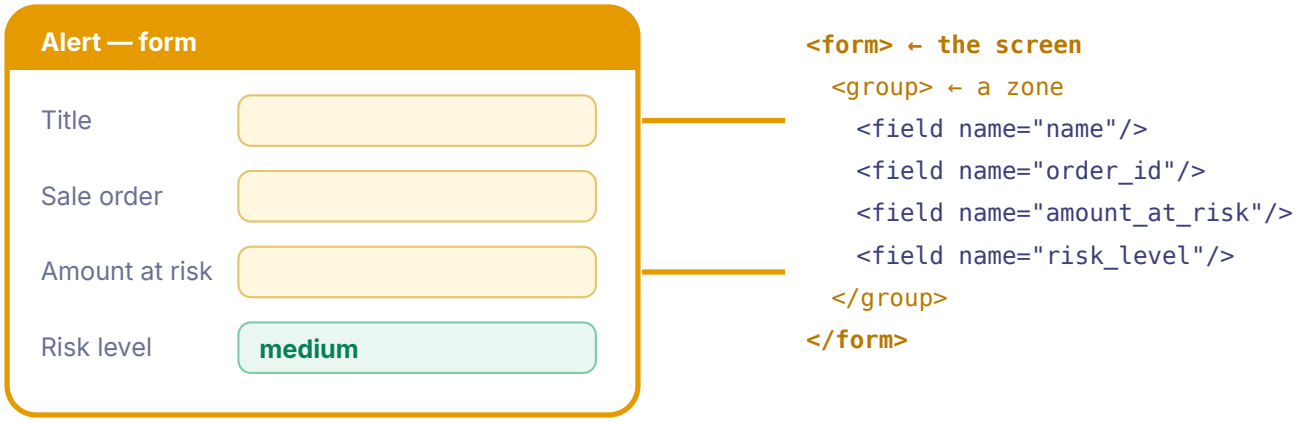
```python
for order in orders:
    print(order.name, order.partner_id.name)
```

B8 Views

An XML file describes each screen

ANALOGY: A HOUSE PLAN

The **model** is the structure (the rooms), the **view** is the display plan (which rooms to show and where). One model can have several views: **form** (record), **list** (table), **kanban** (cards)...



VIEWS/RISK_ALERT_VIEWS.XML

```xml
<record id="alert_view_form" model="ir.ui.view">
    <field name="name">risk.alert.form</field>
    <field name="model">risk.alert</field>
    <field name="arch" type="xml">
        <form>
            <sheet>
                <group>
                    <field name="name"/>
                    <field name="order_id"/>
                    <field name="amount_at_risk"/>
                    <field name="risk_level"/>
                </group>
            </sheet>
        </form>
    </field>
</record>
```

THE LIST VIEW (TABLE)

```xml
<!-- inside the same <record> -->
<list>
    <field name="name"/>
    <field name="amount_at_risk"/>
    <field name="risk_level"/>
</list>
```

ACTION + MENU

```xml
<record id="alert_action" model="ir.actions.act_window">
    <field name="name">Alerts</field>
    <field name="res_model">risk.alert</field>
    <field name="view_mode">list,form</field>
</record>

<menuitem id="alert_menu_root" name="Alerts"
          action="alert_action" sequence="10"/>
```

B8 Views

Change an Odoo screen without touching its code: view inheritance

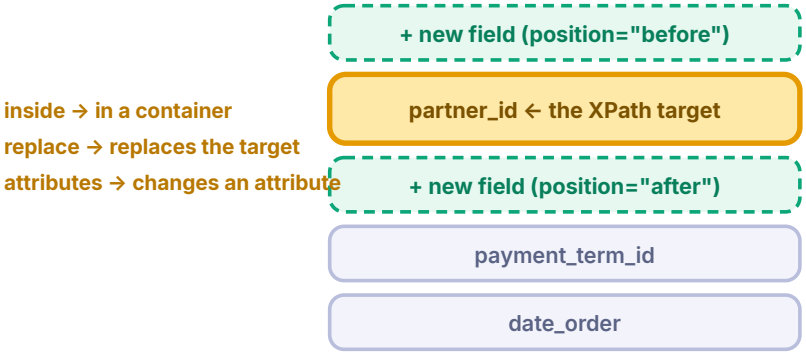
As with models, we never change the original screen: we **target** it with an inheriting view. Two ingredients:

`inherit_id`

Says **which original view** we want to change (its external id, e.g. `sale.view_order_form`).

XPath

An "address" that **locates an element** in the view's XML (e.g. the `partner_id` field).



You aim at an element of the original view, then say where to place the new one

VIEWS/SALE_ORDER_VIEWS.XML

```xml
<record id="sale_order_view_form_risk" model="ir.ui.view">
    <field name="name">sale.order.form.risk</field>
    <field name="model">sale.order</field>
    <field name="inherit_id" ref="sale.view_order_form"/>
    <field name="arch" type="xml">
        <xpath expr="//field[@name='partner_id']" position="after">
            <field name="delivered_not_invoiced"/>
        </xpath>
    </field>
</record>
```

📍 The positions

position	Effect
`after`	Places the new content **right below / after** the target
`before`	Places the new content **before** the target
`inside`	Inserts the content **inside** the targeted element (at the end)
`replace`	**Replaces** the targeted element
`attributes`	Changes the target's **attributes** (e.g. `invisible`)

TRAP

If the id in `ref="…"` or the XPath expression is wrong, Odoo refuses to install the view (error at install). Always check the exact name of the original view and the targeted field.

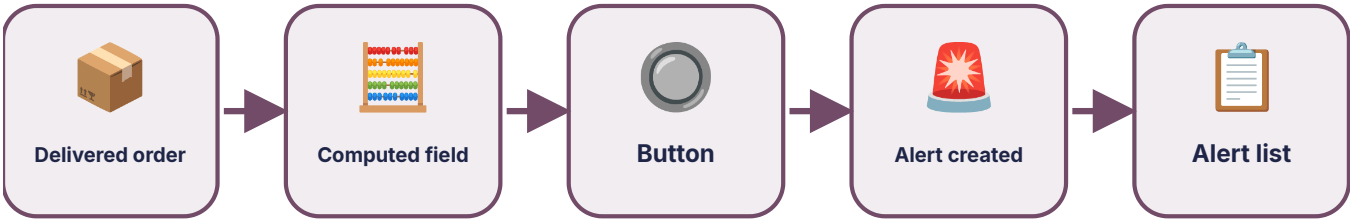
PART 2 · DEVELOPMENT

B9

Mini-project

Assemble all the pieces into a real small module

Every Monday, management asks: *"Which orders did we deliver without invoicing?"* Here is a small real case, which we will build piece by piece.



🪪 1. Manifest

`depends: sale_stock` to reach the order's deliveries.

🧬 2. Extension

`_inherit` on `sale.order` + computed field.

🚨 3. Model + views

`risk.alert`, its access rights, its views, a menu, and an inherited button.

MODELS/SALE_ORDER.PY (1/2)

```python
class SaleOrder(models.Model):
    _inherit = 'sale.order'

    delivered_not_invoiced = fields.Boolean(
        compute='_compute_delivered_not_invoiced', store=True)

    @api.depends('invoice_status', 'picking_ids.state')
    def _compute_delivered_not_invoiced(self):
        for order in self:
            pickings = order.picking_ids.filtered(lambda p: p.state != 'cancel')
            order.delivered_not_invoiced = (
                bool(pickings)
                and all(p.state == 'done' for p in pickings)
                and order.invoice_status == 'to invoice')
```

MODELS/SALE_ORDER.PY (2/2)

```python
    def action_create_alert(self):
        for order in self.filtered('delivered_not_invoiced'):
            self.env['risk.alert'].create({
                'name': f"Delivered but not invoiced: {order.name}",
                'order_id': order.id,
                'amount_at_risk': order.amount_total,
            })
```

B9 Mini-project

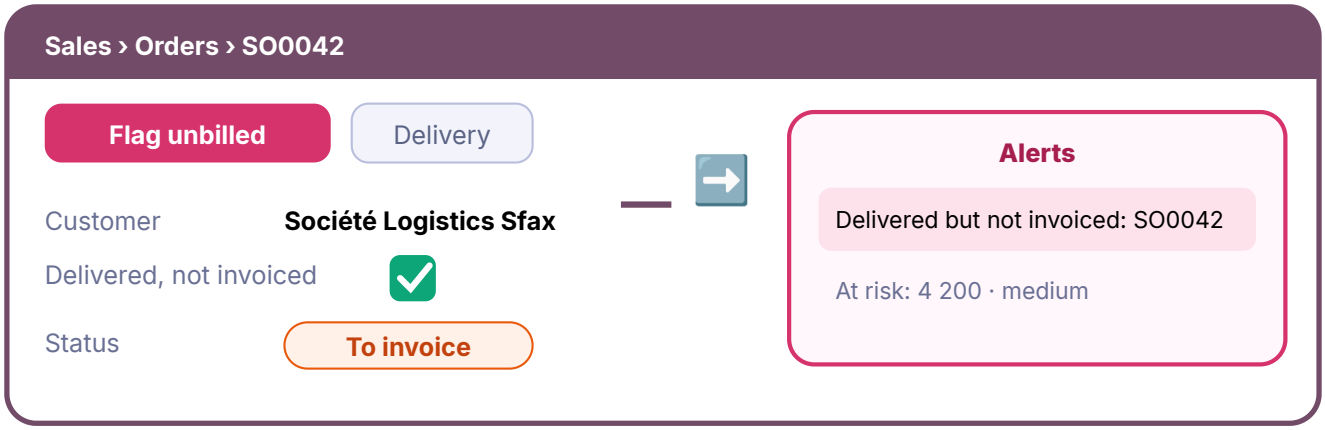
The button, and what you see on screen

🔘 Add the button to the order form

VIEWS/SALE_ORDER_VIEWS.XML (ADD INSIDE THE INHERITANCE)

```
<xpath expr="//header" position="inside">
    <button name="action_create_alert" type="object"
            string="Flag unbilled"
            invisible="not delivered_not_invoiced"/>
</xpath>
```

🖥️ Expected result



🧩 The building blocks used

Block	Guide section
`for`, `lambda`, f-string, dictionary	B1 · Python refresher
Manifest, `depends`, `data`	B2 · An Odoo module
`_inherit`, `Many2one`	B3 · Models and fields
`ir.model.access.csv`	B4 · Access rights
`@api.depends`, `store=True`	B5 · Calculations and business rules
`filtered`, `create`, recordsets	B7 · Searching data
XPath `position`, `inherit_id`, menu, action	B8 · Views

WELL DONE

You just read a small complete Odoo module: a model, an extension, a computation rule, access rights and views. In the next class we will walk this path again together, coding each piece.

SUMMARY SHEET

It all fits on one page

Keep this sheet at hand: it sums up every section.

A1 · The sales workflow

CRM → Sales → Stock → Invoice → Payment. Confirming an order creates the delivery by itself.

A2 · The purchase workflow

Request → Purchase order → Receipt → Vendor bill. Three-way matching: order, receipt, invoice.

A3 · Documents, models and leaks

`sale.order` order, `stock.picking` delivery, `account.move` invoice. Leaks: delivered not invoiced, sleeping stock, excessive discount.

B1 · Python refresher

`class` cutter, `self` "this object", dictionary `{…}`, f-string `f"…"`, `lambda` tiny function.

B2 · An Odoo module

`__manifest__.py` (ID card: `depends`, `data`) + `models/` + `views/` + `security/`.

B3 · Models and fields

`_name` creates · `_inherit` alone extends. Fields: `Char`, `Float`, `Selection`, `Many2one`…

B4 · Access rights

`ir.model.access.csv`: one line per rule; 4 switches `perm_…` (1 = yes, 0 = no). No rule, no access.

B5 · Calculations and business rules

`@api.depends` computes · `@api.constrains` blocks (ValidationError) · `@api.onchange` helps in the form.

B6 · Overriding create() and write()

`create(vals_list)` / `write(vals)` + always `return super()…`

B7 · Searching data

`search`, `search_count`, `mapped`, `filtered`. Domain = list of `(field, operator, value)`.

B8 · Views

`<form>`, `<list>`, action + menu. Inheritance: `inherit_id` + XPath + `position` (after, before, inside, replace).

B9 · Mini-project

Extension of an Odoo model + custom model + rights + views + button = a real module.

WHAT NEXT?

In the next class we will code a complete module from zero. Thanks to this guide, you will already be able to read every line.

GOOD LUCK

Good luck in your exam! 🍀

BONUS · OPTIONAL

Try Odoo 19 online

Without installing anything on your computer

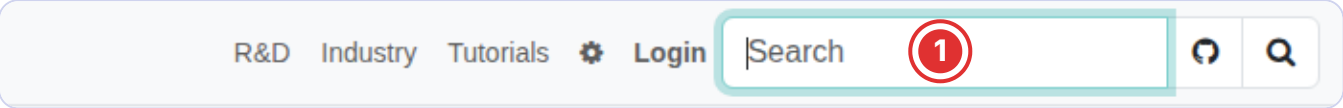
Want to try Odoo yourself? Use **Runbot**, the Odoo team's test platform: it offers ready-to-use instances for every version. For our course it is better than the free trial on odoo.com: you land directly on version **19.0**.

THE LINK

https://runbot.odoo.com

1 **Open Runbot and find the search bar**

It is at the top right of the page.

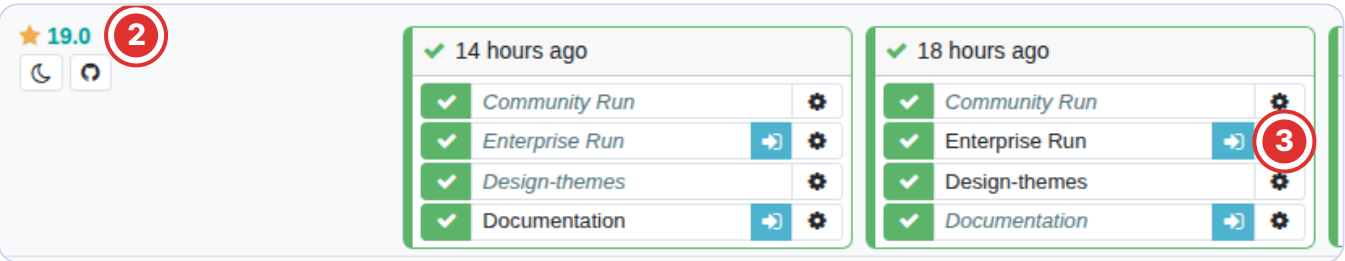


2 **Type 19.0 then Enter, and pick the "19.0" row**

Take the row named **exactly** "19.0" (the first one, with the star). Ignore the rows with very long names: they are work in progress.

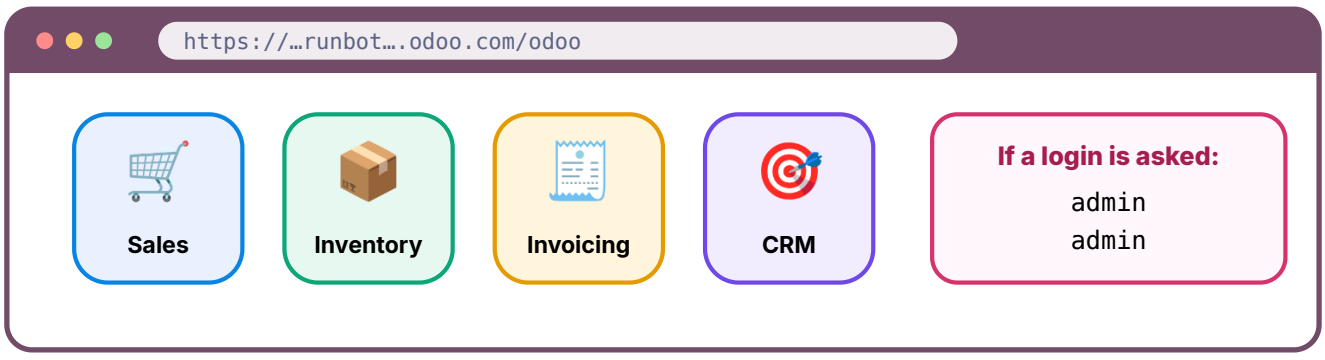
3 **Click a blue connect icon**

Look for the blue icon to the right of "Community Run". If that line has none (as here), use the one next to "Enterprise Run": both open Odoo 19. Try another column if needed.



4 **Odoo opens: you can play!**

If you are asked to log in, use the login `admin` and the password `admin`.



THINGS TO TRY

① **Apps**: install Sales, Inventory and Invoicing.
② Redo the A1 workflow: quotation → confirm → delivery.
③ **Settings → Activate developer mode**, then **Technical → Models**: look for `sale.order` and its fields.

CAREFUL

A Runbot instance is **public and temporary**: it can disappear at any time. Do not enter any personal data.